

HW 4

CSE 331

Professor Kevin Zatloukal

Due: **Saturday** October 26, 2024 11:00pm

Task 1: Off the Beaten Math

(a) `const a = (o: boolean, t: [bigint, bigint]): [bigint, bigint]`

Solution:

$$a : (\mathbb{B}, t : (\mathbb{Z}, \mathbb{Z})) \rightarrow (\mathbb{Z}, \mathbb{Z})$$

$$a(\text{T}, (i, j)) := (i + 1, j)$$

$$a(\text{F}, (i, j)) := (j + 1, i)$$

(b) `const b = (s: bigint, o: boolean): List<bigint>`

Solution:

$$b : (s : \mathbb{Z}, \mathbb{B}) \rightarrow \text{List}(\mathbb{Z})$$

$$b(s, \text{T}) := s :: (-s) :: \text{nil}$$

$$b(s, \text{F}) := (-s) :: s :: \text{nil}$$

(c) `const c = (r: s: bigint, t: [bigint, bigint]): bigint`

Solution:

$$c : \{s : \mathbb{N}, t : (\mathbb{N}, \mathbb{N})\} \rightarrow \mathbb{N}$$

$$c(\{0, (i, j)\}) := i$$

$$c(\{1, (i, j)\}) := i + 1$$

$$c(\{s, (i, j)\}) := j + c(\{s : s - 1, t : t\}) \quad \text{if } s > 1$$

Task 2: Pell on Wheels

- (a) Define two mathematical functions, “p” and “s”, both taking non-negative integers as input. Define each function recursively with pattern matching.

Solution:

$$p : \mathbb{N} \rightarrow \mathbb{N}$$

$$p(0) := 0$$

$$p(1) := 1$$

$$p(n) := p(n-2) + 2p(n-1) \quad \text{if } n > 1$$

$$s : \mathbb{N} \rightarrow \mathbb{N}$$

$$s(0) := 0$$

$$s(n) := s(n-1) + p(n-1) \quad \text{if } n > 0$$

- (b) Show how your mathematical definition would execute $s(6) = 49$

Solution:

$$s(6) = s(5) + p(5)$$

$$p(5) = p(3) + 2p(4)$$

$$p(3) = p(1) + 2p(2) = 1 + 2p(2)$$

$$p(2) = p(0) + 2p(1) = 0 + 2 = 2$$

$$p(3) = 1 + 2p(2) = 1 + 2 \times 2 = 5$$

$$p(4) = p(2) + 2p(3) = 2 + 2 \times 5 = 12$$

$$p(5) = p(3) + 2p(4) = 5 + 2 \times 12 = 29$$

$$s(5) = s(4) + p(4) = s(4) + 12$$

$$s(4) = s(3) + p(3) = s(3) + 5$$

$$s(3) = s(2) + p(2) = s(2) + 2$$

$$s(2) = s(1) + p(1) = s(1) + 1$$

$$s(1) = s(0) + p(0) = 0 + 0 = 0$$

$$s(2) = s(1) + 1 = 0 + 1 = 1$$

$$s(3) = s(2) + 2 = 1 + 2 = 3$$

$$s(4) = s(3) + 5 = 3 + 5 = 8$$

$$s(5) = s(4) + 12 = 8 + 12 = 20$$

$$s(6) = s(5) + p(5) = 20 + 29 = 49$$

Task 3: Good To the Last Swap

(a) formalize “ns”

Solution:

$$\begin{aligned} ns &: \mathbb{Z} \rightarrow \mathbb{Z} \\ ns(j) &:= 18 - j && \text{if } 0 \leq j < 5 \text{ or } 13 \leq j < 19 \\ ns(j) &:= 31 - j && \text{if } 6 \leq j < 13 \text{ or } 19 \leq j < 26 \\ ns(j) &:= j && \text{otherwise} \end{aligned}$$

(b) formalize “cipher”

Solution:

$$\begin{aligned} \text{cipher} &: \text{List}(\mathbb{Z}) \rightarrow \text{List}(\mathbb{Z}) \\ \text{cipher}(\text{nil}) &:= \text{nil} \\ \text{cipher}(j :: L) &:= ns(j) :: \text{cipher}(L) \end{aligned}$$

Task 4: Mazed and Confused

(a) Formalize “mazeA”, “mazeB”, and “mazeC”

Solution:

mazeA: Color $\rightarrow$ Maze

$r1(c)$: Color $\rightarrow$ Row := rcons({form: ANGLED, color: c , direction: BR}, rcons({form: ANGLED, color: c , direction: BL}, rnil))

$r2(c)$: Color $\rightarrow$ Row := rcons({form: ANGLED, color: c , direction: TL}, rcons({form: ANGLED, color: c , direction: TR}, rnil))

mazeA(c) := mcons($r1(c)$, mcons($r2(c)$, mcons($r1(c)$, mcons($r2(c)$, mnil))))

mazeB: Color $\rightarrow$ Maze

$r3(c)$: Color $\rightarrow$ Row := rcons({form: ANGLED, color: c , direction: TR}, rcons({form: ANGLED, color: c , direction: BL}, rnil))

$r4(c)$: Color $\rightarrow$ Row := rcons({form: ANGLED, color: c , direction: BR}, rcons({form: ANGLED, color: c , direction: TL}, rnil))

mazeB(c): Maze = mcons($r3(c)$, mcons($r4(c)$, mcons($r3(c)$, mcons($r4(c)$, mnil))))

mazeC: Color $\rightarrow$ Maze

$r5(c)$: Color $\rightarrow$ Row := rcons({form: ANGLED, color: c , direction: BR}, rcons({form: STRAIGHT, color: c , direction: RL}, rnil))

$r6(c)$: Color $\rightarrow$ Row := rcons({form: ANGLED, color: c , direction: TR}, rcons({form: ANGLED, color: c , direction: BL}, rnil))

$r7(c)$: Color $\rightarrow$ Row := rcons({form: STRAIGHT, color: c , direction: RL}, rcons({form: ANGLED, color: c , direction: TL}, rnil))

mazeC(c): Maze = mcons($r5(c)$, mcons($r6(c)$, mcons($r7(c)$, mcons($r5(c)$, mnil))))

(b) Define recursive functions that repeat these designs to allow for creating larger mazes.

Solution: Using $r1(c)$ through $r7(c)$ from (a) for a clean answer, we have the following three mazes:

mazeA: (Color, $\mathbb{N}$) $\rightarrow$ Maze

mazeA($c, 0$) := mnil

mazeA(c, n) := mcons($r1(c)$, mcons($r2(c)$, mazeA($c, n - 2$))) if $n > 0$

mazeB: (Color, $\mathbb{N}$) $\rightarrow$ Maze

mazeB($c, 0$) := mnil

mazeB(c, n) := mcons($r3(c)$, mcons($r4(c)$, mazeB($c, n - 2$))) if $n > 0$

mazeC: (Color, $\mathbb{N}$) $\rightarrow$ Maze

mazeC($c, 0$) := mnil

mazeC(c, n) := mcons($r5(c)$, mcons($r6(c)$, mcons($r7(c)$, mazeC($c, n - 3$)))) if $n > 0$

Task 5: My Flips are Sealed

(a) formalize “bflip-vert”

Solution:

bflip-vert: Block $\rightarrow$ Block

bflip-vert(b) :=

$$\begin{cases} b & \text{if } b.\text{form} = \text{STRAIGHT} \\ \{\text{form: ANGLED, color: } c, \text{ direction: BR}\} & \text{if } b = \{\text{form: ANGLED, color: } c, \text{ direction: TR}\} \\ \{\text{form: ANGLED, color: } c, \text{ direction: BL}\} & \text{if } b = \{\text{form: ANGLED, color: } c, \text{ direction: TL}\} \\ \{\text{form: ANGLED, color: } c, \text{ direction: TR}\} & \text{if } b = \{\text{form: ANGLED, color: } c, \text{ direction: BR}\} \\ \{\text{form: ANGLED, color: } c, \text{ direction: TL}\} & \text{if } b = \{\text{form: ANGLED, color: } c, \text{ direction: BL}\} \end{cases}$$

(b) write out “rflip-vert” that flips a row

Solution:

r	rflip-vert (r)
rnil	rnil
rcons(a , rn timer)	rcons($bv(a)$, rn timer)
rcons(a , rcons(b , rn timer))	rcons($bv(a)$, rcons($bv(b)$, rn timer))
rcons(a , rcons(b , rcons(c , rn timer)))	rcons($bv(a)$, rcons($bv(b)$, rcons($bv(c)$, rn timer)))
...	

(c) formalize “rflip-vert”

Solution:

rflip-vert: Row $\rightarrow$ Row

rflip-vert(rnil) := rn timer
rflip-vert(rcons(a , R)) := rcons($bv(a)$, rflip-vert(R))

(d) write out “mflip-vert” that flips a maze

Solution:

r	mflip-vert (r)
mnil	mnil
mcons(u , mnil)	mcons($r(u)$, mnil)
mcons(u , mcons(v , mnil))	mcons($r(v)$, mcons($r(u)$, mnil))
mcons(u , mcons(v , mcons(w , mnil)))	mcons(w , mcons(v , mcons(u , mnil)))
...	

(e) formalize “mflip-vert”

Solution:

mflip-vert: Maze $\rightarrow$ Maze

mflip-vert(mnil) := mnil
mflip-vert(mcons(u , R)) := mconcat(mflip-vert(R), mcons($r(u)$, mnil))